NOTE You should use the User Secrets provider only in development, not in production, so in the preceding snippet you add the provider conditionally to `ConfigurationManager`. In production you should use environment variables or Azure Key Vault, as discussed earlier. All this is configured correctly by default when you use the default `WebApplicationBuilder`.

The `AddUserSecrets` method has several overloads, but the simplest is a generic method that you can call by passing your application's `Program` class as a generic argument, as shown in the preceding example. The User Secrets provider needs to read the `UserSecretsId` property that you (or Visual Studio) added to the .csproj file. The `Program` class acts as a simple marker to indicate which assembly contains this property.

NOTE If you're interested, the .NET SDK uses the `UserSecretsId` property in your .csproj file to generate an assembly-level `UserSecretsIdAttribute`. Then the provider reads this attribute at runtime to determine the `UserSecretsId` of the app and generates the path to the secrets.json file.

And there you have it—safe storage of your secrets outside your project folder during development. This cautious approach may seem like overkill, but if you have anything you consider to be remotely sensitive that you need to load into configuration, I strongly urge you to use environment variables or User Secrets.

It's almost time to leave configuration providers behind, but before we do, I'd like to show you the ASP.NET Core configuration system's party trick: reloading files on the fly.

10.2.4 Reloading configuration values when they change